UD1. INTRODUCCIÓN



QUÉ VAMOS A VER EN ESTE TEMA

1. Introducción
2. Programas y programación
3. Paradigmas de programación
4. Fases de programación
5. Ciclo de vida del software
6. Pseudocódigo y diagramas de flujo

1. INTRODUCCIÓN

- ¿Cuántas acciones de las que has realizado hoy, crees que están relacionadas con la programación?

Vivímos en una época tecnológica, por tanto, la mayoría de nuestras acciones están relacionadas con la programación.

En este tema realizaremos un recorrido por los conceptos fundamentales de la programación de aplicaciones.

2. PROGRAMAS Y PROGRAMACIÓN

Utilizamos la programación para resolver un problema que se nos plantea. Esto tiene cierta analogía con el mundo real.

Resolución de problemas	
En la vida real...	**En Programación...**
Observación de la situación o problema.	**Análisis del problema:** requiere que el problema sea definido y comprendido claramente para que pueda ser analizado con todo detalle.
Pensamos en una o varias posibles soluciones.	**Diseño o desarrollo de algoritmos:** procedimiento paso a paso para solucionar el problema dado.
Aplicamos la solución que estimamos más adecuada.	**Resolución del algoritmo elegido en la computadora:** consiste en convertir el algoritmo en programa, ejecutarlo y comprobar que soluciona verdaderamente el problema.

2. PROGRAMAS Y PROGRAMACIÓN

Nuestra solución al problema debe ser **correcto y eficaz.**

Para conseguirlo, nuestra solución tendrá que tener en cuenta los siguientes conceptos:

1.Abstracción: se trata de realizar un análisis del problema para descomponerlo en problemas más pequeños y de menor complejidad, describiendo cada uno de ellos de manera precisa.

2.Divide y vencerás, esta suele ser considerada una filosofía general para resolver problemas y de aquí que su nombre no sólo forme parte del vocabulario informático

3.Encapsulación: consiste en ocultar la información para poder implementarla de diferentes maneras sin que esto influya en el resto de elementos.

4.Modularidad: estructuramos cada parte en módulos independientes, cada uno de ellos tendrá su función correspondiente.

2. PROGRAMAS Y PROGRAMACIÓN

2.1. ALGORITMOS Y PROGRAMAS

Para la resolución de problemas se emplean los algoritmos.

¿Qué es un algoritmo?

Secuencia ordenada de pasos, descrita sin ambigüedades, que conducen a la solución de un problema dado.

Los algoritmos son independiente del lenguaje de programación utilizado

2. PROGRAMAS Y PROGRAMACIÓN

2.1. ALGORITMOS Y PROGRAMAS

Características de un algoritmo

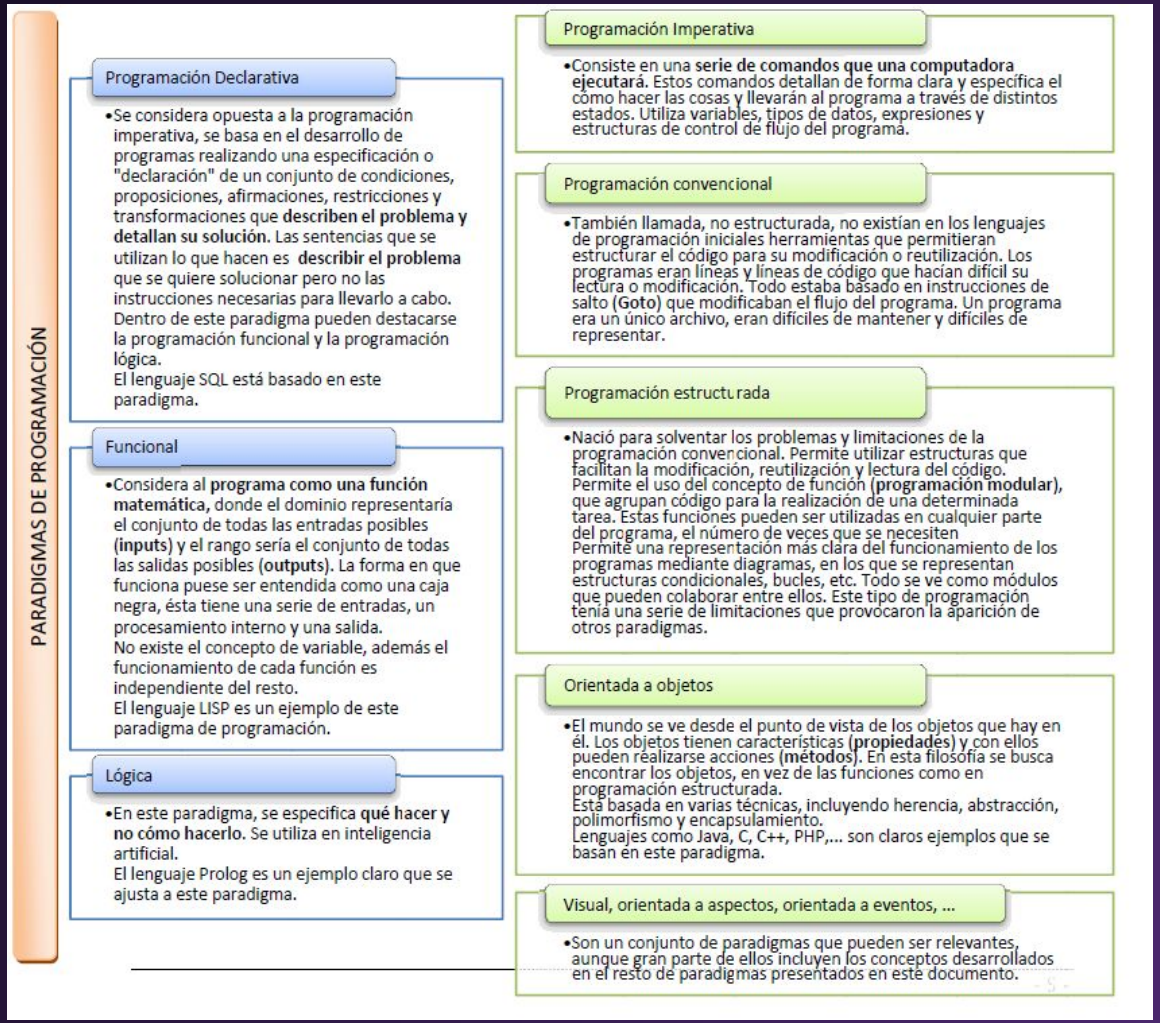
- Debe ser preciso e indicar el orden de realización paso a paso.
- Debe estar definido, si se ejecuta dos o más veces, debe obtener el mismo resultado cada vez.
- Debe ser finito, debe tener un número finito de pasos.

3. PARADIGMAS DE PROGRAMACIÓN

Para resolver un problema existen multitud de formas. Estas formas de solucionarlos es lo que se denomina paradigma.

Paradigma de programación: es un modelo básico para el diseño y la implementación de programas. Este modelo determinará cómo será el proceso de diseño y estructura final del programa.

El paradigma representa un enfoque para la construcción del software. Cada uno tendrá sus ventajas e inconvenientes, será más o menos apropiado, pero no es correcto decir que exista uno mejor que otro.


PARADIGMAS DE PROGRAMACIÓN
Programación Declarativa
•Se considera opuesta a la programación imperativa, se basa en el desarrollo de programas realizando una especificación o "declaración" de un conjunto de condiciones, proposiciones, afirmaciones, restricciones y transformaciones que describen el problema y detallan su solución. Las sentencias que se utilizan lo que hacen es describir el problema que se quiere solucionar pero no las instrucciones necesarias para llevarlo a cabo. Dentro de este paradigma pueden destacarse la programación funcional y la programación lógica.
El lenguaje SQL está basado en este paradigma.
Funcional
•Considera al programa como una función matemática, donde el dominio representaría el conjunto de todas las entradas posibles (inputs) y el rango sería el conjunto de todas las salidas posibles (outputs). La forma en que funciona puese ser entendida como una caja negra, ésta tiene una serie de entradas, un procesamiento interno y una salida.
No existe el concepto de variable, además el funcionamiento de cada función es independiente del resto.
El lenguaje LISP es un ejemplo de este paradigma de programación.
Lógica
•En este paradigma, se especifica qué hacer y no cómo hacerlo. Se utiliza en inteligencia artificial.
El lenguaje Prolog es un ejemplo claro que se ajusta a este paradigma.
Programación Imperativa
•Consiste en una serie de comandos que una computadora ejecutará. Estos comandos detallan de forma clara y específica el cómo hacer las cosas y llevarán al programa a través de distintos estados. Utiliza variables, tipos de datos, expresiones y estructuras de control de flujo del programa.
Programación convencional
•También llamada, no estructurada, no existían en los lenguajes de programación iniciales herramientas que permitieran estructurar el código para su modificación o reutilización. Los programas eran líneas y líneas de código que hacían difícil su lectura o modificación. Todo estaba basado en instrucciones de salto (Goto) que modificaban el flujo del programa. Un programa era un único archivo, eran difíciles de mantener y difíciles de representar.
Programación estructurada
•Nació para solventar los problemas y limitaciones de la programación convencional. Permite utilizar estructuras que facilitan la modificación, reutilización y lectura del código. Permite el uso del concepto de función (programación modular), que agrupan código para la realización de una determinada tarea. Estas funciones pueden ser utilizadas en cualquier parte del programa, el número de veces que se necesiten
Permite una representación más clara del funcionamiento de los programas mediante diagramas, en los que se representan estructuras condicionales, bucles, etc. Todo se ve como módulos que pueden colaborar entre ellos. Este tipo de programación tenía una serie de limitaciones que provocaron la aparición de otros paradigmas.
Orientada a objetos
•El mundo se ve desde el punto de vista de los objetos que hay en él. Los objetos tienen características (propiedades) y con ellos pueden realizarse acciones (métodos). En esta filosofía se busca encontrar los objetos, en vez de las funciones como en programación estructurada.
Está basada en varias técnicas, incluyendo herencia, abstracción, polimorfismo y encapsulamiento.
Lenguajes como Java, C, C++, PHP,... son claros ejemplos que se basan en este paradigma.
Visual, orientada a aspectos, orientada a eventos, ...
•Son un conjunto de paradigmas que pueden ser relevantes, aunque gran parte de ellos incluyen los conceptos desarrollados en el resto de paradigmas presentados en este documento.

4. Fases de la programación

El proceso de creación de software puede dividirse en diferentes fases:

1. Fase de resolución del problema: Análisis y diseño
2. Fase de implementación: Implementación y pruebas
3. Fase de explotación y mantenimiento.

5 .Ciclo de vida del software

Normalmente se sigue un modelo en cascada con las siguientes fases:

1. Análisis
2. Diseño
3. Implementación
4. Pruebas
5. Documentación
6. Puesta en producción



6. PSEUDOCÓDIGO Y DIAGRAMAS DE FLUJO

Programas que se van a utilizar:

- **PSeInt:** **http://pseint.sourceforge.net/**
- **DrawIO:** **https://app.diagrams.net/**

1 ¿Qué es pseudocódigo?

Esta técnica se basa en el uso de palabras clave en lenguaje natural, constantes (Estructura de datos que se utiliza en los lenguajes de programación que no puede cambiar su contenido en el transcurso del programa.), variables (Estructura de datos que, como su nombre indica, puede cambiar de contenido a lo largo de la ejecución de un programa.), otros objetos, instrucciones y estructuras de programación que expresan de forma escrita la solución del problema. **Es la técnica más utilizada actualmente.**



1 ¿Cómo se escribe en pseudocódigo?

Se utiliza el siguiente vocabulario:

Escribir por pantalla: Escribir/Mostrar

Leer por pantalla: Leer

Después de cada instrucción se pone ;



1 ¿Cómo se escribe en pseudocódigo?

Una vez que se tiene el algoritmo en pseudocódigo es muy fácil pasarlo al lenguaje de programación.

Estructura de un programa en pseudocódigo:

```
1   Algoritmo primerAlgoritmo
2       Escribir "Introduce el primer número";
3       Leer numero1;
4       Escribir "Introduce el segundo número";
5       Leer numero2;
6       //Para asignar a una variable un valor se utiliza <-
7       resultado ← numero1 + numero2;
8       Mostrar  "El resultado es:" resultado;
9
10  FinAlgoritmo
11
```

1.Ejemplo de pseudocódigo

Pedir al usuario la edad y escribirla por pantalla:

```
1  Algoritmo AlgoritmoEDAD
2      Escribir "¿Cuál es tu edad?";
3      Leer edad;
4      Escribir  "Tu edad es:"+ edad;
5  FinAlgoritmo
6
```

2 .Operadores matemáticos y lógicos

Se pueden utilizar símbolos matemáticos y operadores lógicos para realizar operaciones

Operación	Sgn	Descripción
Suma	+	Adiciona una o mas variables
Resta	-	Sustrae una o mas variables
Multiplicación	*	Multiplica el valor de dos o mas variables
División	/	Divide el valor de dos o mas variables
Entero de división	\	Devuelve la parte entera de la división entre dos números
Potencia	^	X^n El valor x elevado a la n
Modulo	Mod	Devuelve el resto de la división de dos números

2 .Operadores matemáticos y lógicos

Se pueden utilizar símbolos matemáticos y operadores lógicos para realizar operaciones

Operador relacional	*Significado*	*Ejemplo*
>	Mayor que	3>2
<	Menor que	"ABC"<"abc"
=	Igual que	4=3
<=	Menor o igual que	"a"<="b"
>=	Mayor o igual que	4>=5

3. Definición de variables

En la mayoría de los lenguajes de programación más utilizados las variables tienen asociado un tipo de dato. Pueden ser datos simples o compuestos.



```
Definir <var1> , <var2> , ... , <varN>  Como [REAL/ENTERO/LOGICO/CARACTER];
```

Tipos simples

```
Dimension edad[12];
```

Datos compuestos (arrays). Se define el tipo NombreVble[tamaño]

RESUMEN DE COMANDOS

- Todas las instrucciones van entre **Algoritmo** NombreAlgoritmo y **FinAlgoritmo**

```
1  Algoritmo AlgoritmoEDAD
2
3      //Instrucciones
4  FinAlgoritmo
5
```

- Dar información al usuario: **Escribir**
- Recoger información del usuario: **Leer**
- Asignar un valor: **<-**

```
1  Algoritmo AlgoritmoEDAD
2      Definir numX Como Entero;
3      Definir numY Como Entero;
4      Escribir "Dime un número";
5      Leer numX;
6      numY←2*numX+3*numX+4;
7      Escribir "El resultado es " numY
8  FinAlgoritmo
9
```

- Definir una variable simple: **Definir** nombreVble **como** tipoVble
- Definir una variable compuesta (arrays): **Dimension** nombreVble **[tamaño]**
- **Todas las sentencias tienen que acabar en ;**

Ejercicios



- Pseudocódigo que calcule la suma de dos números introducidos por teclado
- Pseudocódigo que pida el peso y la edad y la muestre por pantalla
- Pseudocódigo que escriba el área de un círculo
- Pseudocódigo que escriba el área de un triángulo
- Pseudocódigo que escriba el volumen de un cilindro
- Elaborar un algoritmo que obtenga e imprima el valor de Y a partir de la ecuación: Y= 3X + 2 + 7X – 15, introduciendo el valor que queramos de X.
- Crear un pseudocódigo que proporcione el precio medio de un producto, calculado a partir del precio del mismo en tres establecimientos distintos.

3. Sentencias condicionales IF y IF/ELSE

Se usan para ejecutar un trozo de código si se cumple una condición

```
Si expresion_logica Entonces
    acciones_por_verdadero
SiNo
    acciones_por_falso
Fin Si
```

La condición siempre tiene que devolver un valor lógico (True/False)



```
Si expresion_logica Entonces
    acciones_por_verdadero
Fin Si
```

3. Sentencias condicionales IF y IF/ELSE

La expresión lógica puede ser una expresión simple o compleja.

```
Si numUsuario<0 Entonces
```

Simple: Se evalúa sólo un valor

```
Si numUsuario>0  Y numUsuario<5 Entonces
```

```
Si numUsuario>0  O numUsuario<5 Entonces
```

Compuesta: Se evalúan varios valores

AND

x_1	x_2	$x_1 \wedge x_2$
1	1	1
1	0	0
0	1	0
0	0	0

OR

x_1	x_2	$x_1 \vee x_2$
1	1	1
1	0	1
0	1	1
0	0	0

5. Ejemplos sentencias condicionales e iterativas

Condicionales: saber si un número es mayor que cero

```
Algoritmo MayorQueCero
    Definir numUsuario Como Entero;

    Escribir "Dime un número";
    Leer numUsuario;
    Si numUsuario>0 Entonces
        Escribir "Tu número es mayor que cero";
    SiNo
        Escribir "Tu número es menor que cero";
    Fin Si
FinAlgoritmo
```



Ejercicios condicionales

- Realizar el pseudocódigo de un programa que permita saber si un número es mayor, menor o igual a cero.
- Pseudocódigo que pida dos número y diga cual es el mayor
- Pseudocódigo que implemente esta función:

$$f(x) = \begin{cases} 0 & x < 0 \\ x & 0 \leq x \leq 1 \\ 2 - x & 1 < x \leq 2 \\ 0 & x > 2 \end{cases}$$

- Pseudocódigo que pida un mes, en número y diga si el mes tiene 30, 31 ó 28 días
- Pseudocódigo que pida un número y diga si es par o impar

4. Sentencias iterativas (FOR y WHILE)

Permiten repetir una acción mientras se cumpla una condición

```
Para variable_numerica<-valor_inicial Hasta valor_final Con Paso paso Hacer
    secuencia_de_acciones
Fin Para
```

```
Mientras expresion_logica Hacer
    secuencia_de_acciones
Fin Mientras
```



5. Ejemplos sentencias condicionales e iterativas

Iterativas: pedir números al usuario hasta que introduzca cero

```
Algoritmo MayorQueCero
    Definir numUsuario Como Entero;

    Escribir "Dime un número";
    Leer numUsuario;
    Mientras numUsuario≠0 Hacer
        Escribir "Dime un número";
        Leer numUsuario;
    Fin Mientras
FinAlgoritmo
```



Ejercicios iteraciones



- Escribe el pseudocódigo que pida un número y muestre por pantalla los números hasta el 0 desde el introducido
- Pseudocódigo que pida un número y muestre su tabla de multiplicar
- Pseudocódigo que muestre por pantalla el doble del número introducido por el usuario hasta que introduzca un número menor que 0
- Pseudocódigo que calcule la suma de todos los números entre 1 y 20 que son múltiplos de 5 o de 7. Para comprobar que X es múltiplo de Y basta con comprobar si el resto de la división entera de X entre Y es 0.
- **HACER: Escribe el pseudocódigo que pida un número y muestre por pantalla la suma de ese número hasta 0**

DIAGRAMAS DE FLUJO



6 ¿Qué es un diagrama de flujo?

- Permite representar la secuencia lógica para representar una tarea
- Antes de realizar el diagrama de flujo hay que realizar el algoritmo



6 Ejemplo diagrama de flujo

Cocinar un huevo para otra persona sería:

Algoritmo:



- Pregunto si quiere el huevo frito.

- Si me dice que si, lo frio, si me dice que no, lo hago hervido.

- Una vez cocinado le pregunto si quiere sal en el huevo.

- Si me dice que no, lo sirvo en el Plato, si me dice que si, le hecho sal y después lo sirvo en el plato.

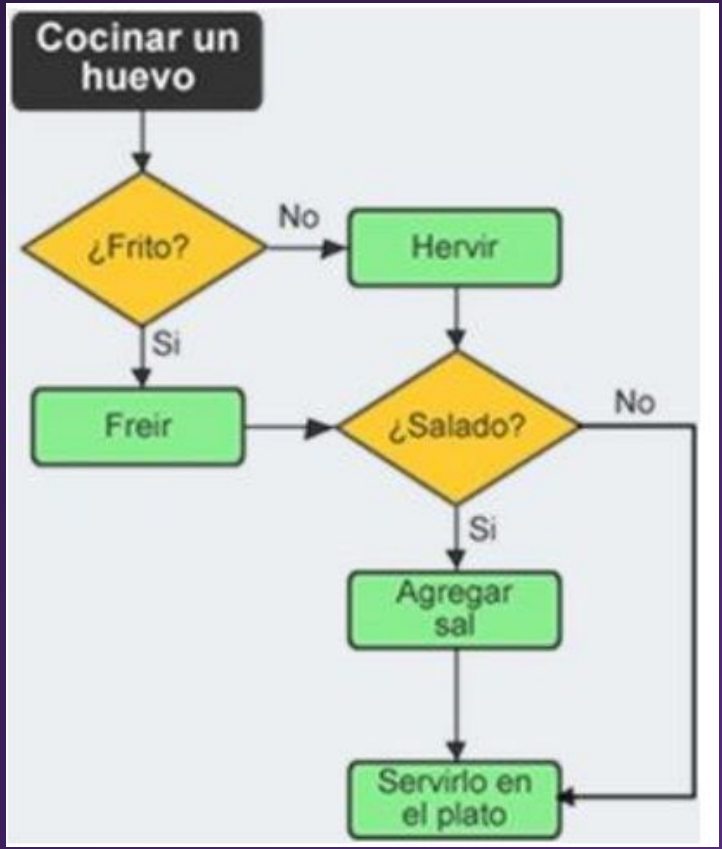
6 Ejemplo diagrama de flujo

Diagrama de flujo:

-Los pasos no pueden cambiar su posición.

Sería imposible preguntarle si lo quiere frito después de haberlo hervido, por ejemplo.

Es muy importante que los pasos sean una secuencia lógica y ordenada.



7. Reglas y símbolos diagrama de flujo

1. Todos los símbolos han de estar conectados

2. A un símbolo de proceso pueden llegarle varias líneas

3. A un símbolo de decisión pueden llegarle varias líneas, pero sólo saldrán dos (Si o No, Verdadero o Falso).

4. A un símbolo de inicio nunca le llegan líneas.

5. De un símbolo de fin no parte ninguna línea.



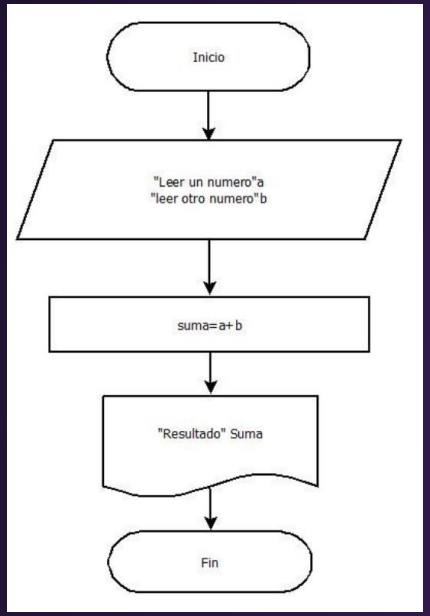
7. Reglas y símbolos diagrama de flujo

Símbolo	Nombre	Función
	Inicio / Final	Representa el inicio y el final de un proceso
	Linea de Flujo	Indica el orden de la ejecución de las operaciones. La flecha indica la siguiente instrucción.
	Entrada / Salida	Representa la lectura de datos en la entrada y la impresión de datos en la salida
	Proceso	Representa cualquier tipo de operación
	Decisión	Nos permite analizar una situación, con base en los valores verdadero y falso



7. Ejemplos diagramas de flujo

Programa informático que nos sume dos número y nos dé el resultado en pantalla.

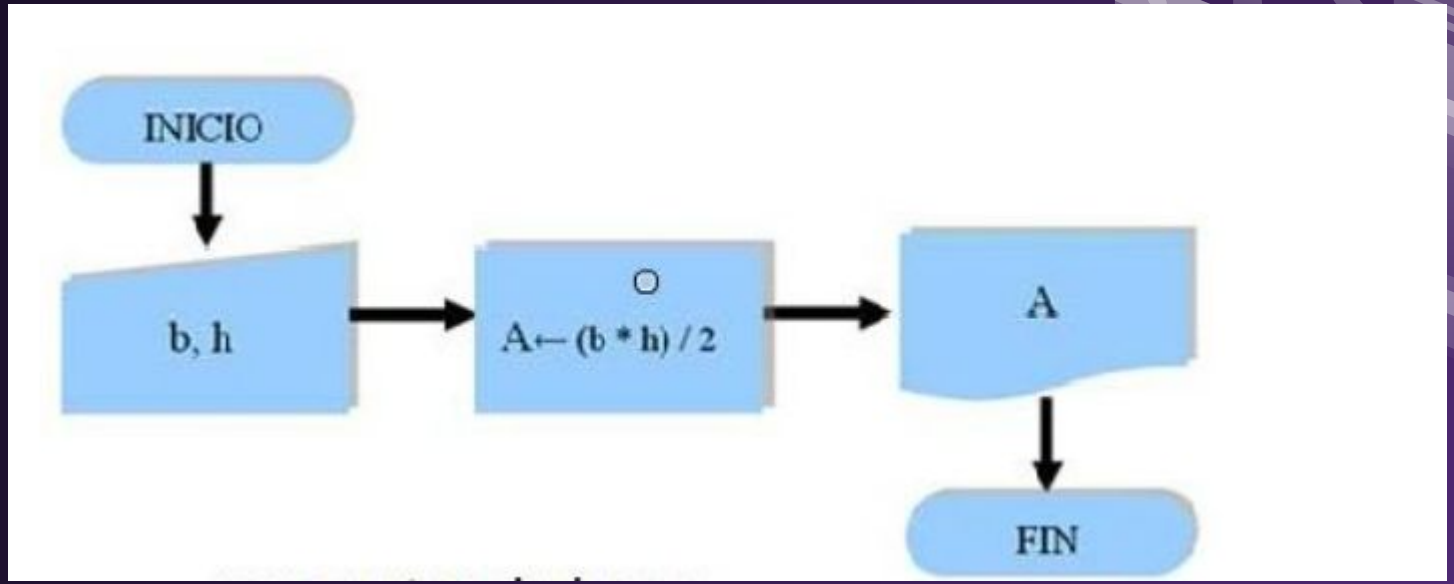


A veces se utiliza este símbolo cuando se muestra por pantalla el resultado:



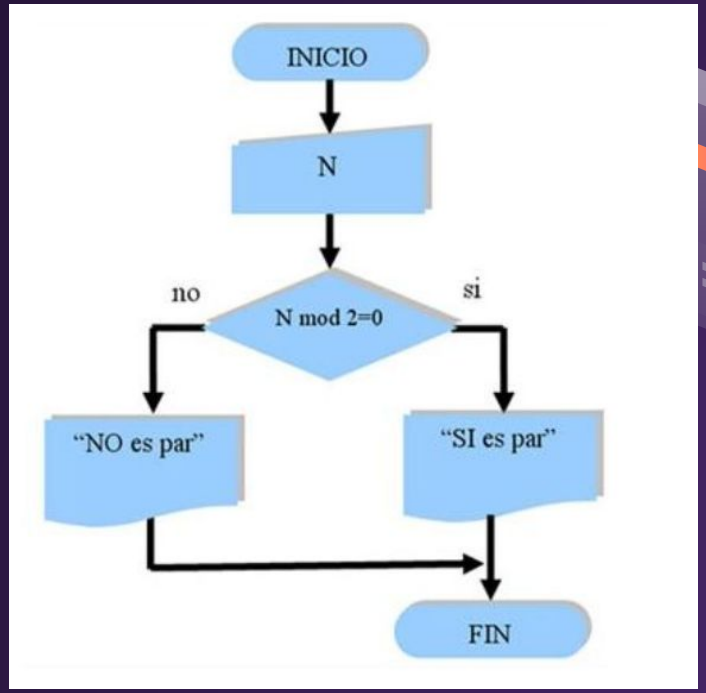
7. Ejemplos diagramas de flujo

Área de un triángulo



7. Ejemplos diagramas de flujo

Número par e impar



Ejercicios diagramas

- Realiza el diagrama que represente: pedir dos números por teclado,sumarlos y mostrar al usuario el resultado
- Hacer un diagrama de flujo que permita leer 2 números diferentes y nos diga cual es el mayor de los 2 números.
- Hacer un diagrama de flujo que permita escribir los 20 primeros pares.
- Hacer un diagrama que lea N números, calcule y escriba la suma de los pares y el producto de los impares.
- Un año es bisiesto si es múltiplo de 4, exceptuando los múltiplos de 100, que sólo son bisiestos cuando son múltiplos además de 400, por ejemplo el año 1900 no fue bisiesto, pero el año 2000 sí lo es. Hacer un diagrama que dado un año A nos diga si es o no bisiesto.

